

Airport Management System: Relational to MongoDB Migration

Final Project Report

NoSQL Database Course Project

Repository: <https://github.com/Ioni007/airport-nosql-project>

1. Introduction

This project demonstrates an end-to-end migration from a relational SQL Server Airport Management System database to MongoDB. The work includes relational schema design, large synthetic data population, NoSQL data modeling, programmatic migration with transformations, validation, visualization, Docker-based database services, and final review documentation.

The project domain is airport management. The relational database contains airlines, airplanes, aircraft models, flights, clients, employers, travel agencies, tickets, reservations, and maintenance care records. MongoDB was chosen as the NoSQL target because the final workload is reporting-oriented and benefits from denormalized documents with embedded snapshots and precomputed fields.

2. Relational Database Design

The SQL Server database is named AirportManagement. The schema is implemented in `sql/schema.sql` and contains ten relational tables. The ER diagram is documented in `docs/er_diagram.md` with primary keys, foreign keys, and cardinalities.

Table	Purpose
AIRLINE	Airline company metadata.
MODEL	Aircraft model metadata.
AIRPLANE	Physical airplanes assigned to airlines and models.
FLIGHT	Scheduled flights operated by airlines.
CLIENT	Passengers/customers.
EMPLOYER	Staff members involved in operations or maintenance.
TRAVELAGENCY	Agencies that can create reservations.
TICKET	Flight tickets and seat assignments.
RESERVE	Reservation transactions.
CARE	Maintenance/service records.

The schema includes primary keys on every table, foreign keys for all relationships, NOT NULL constraints, unique constraints, and multiple CHECK constraints for business rules such as valid statuses, positive prices, different origin/destination airports, and valid reservation agency usage.

3. Data Population

Baseline seed data is implemented in `sql/seed.sql`. Large synthetic data is generated by `sql/generate_fake_data.py` using Faker. The generated SQL is stored in `sql/generated_seed.sql`.

Entity	Record Count
AIRLINE	4
MODEL	4
CLIENT	2,005
TRAVELAGENCY	3
AIRPLANE	5
EMPLOYER	4
FLIGHT	805
TICKET	15,007
RESERVE	12,005
CARE	303

The TICKET and RESERVE tables both exceed the 10,000-record requirement while preserving relational integrity.

4. Choice of MongoDB

MongoDB was selected because the target workload is document-oriented reporting and visualization. Common airport questions require joined data in the relational model, while MongoDB allows the final reporting views to be stored as denormalized documents with embedded snapshots and derived fields.

MongoDB supports nested document structures, indexes for idempotent upserts, Python integration through PyMongo, and convenient visualization workflows.

5. Comparison with Redis and Neo4j

Redis is a key-value database optimized for caching, counters, queues, and direct lookups. It would be fast for keys such as airline:1 or client:300, but it is less natural for rich nested reservation documents and validation-oriented reporting queries.

Neo4j is a graph database. It could model Client-MADE-Reservation-FOR-Ticket-ON-Flight relationships, but the project focuses on denormalized reporting documents, validation checks, and dashboards. MongoDB is therefore the simpler and more suitable target.

```
(:Client)-[:MADE]->(:Reservation)-[:FOR]->(:Ticket)-[:ON]->(:Flight)
(:Flight)-[:OPERATED_BY]->(:Airline)
(:Airplane)-[:OWNED_BY]->(:Airline)
(:TravelAgency)-[:CREATED]->(:Reservation)
```

6. NoSQL Database Modeling

Collection	Source Tables	Purpose
airlines	AIRLINE, FLIGHT, AIRPLANE	Airline performance and fleet counts.
airplanes	AIRPLANE, AIRLINE, MODEL	Fleet lookup with embedded airline/model details.
flights	FLIGHT, AIRLINE, TICKET, RESERVE	Flight schedule and reservation activity.
clients	CLIENT, RESERVE, TICKET	Customer reservation profile.

Collection	Source Tables	Purpose
travel_agencies	TRAVELAGENCY, RESERVE, TICKET	Agency analytics.
reservations	RESERVE, CLIENT, TICKET, FLIGHT, AIRLINE, TRAVELAGENCY	Denormalized reservation records.
maintenance	CARE, AIRPLANE, EMPLOYER	Maintenance history.
migration_runs	Runtime metadata	Migration audit and idempotency evidence.

Embedding is used for stable, compact data that is frequently read together, such as airplane airline/model snapshots and reservation client/ticket/flight/agency snapshots. Large one-to-many relationships are not fully embedded; instead, aggregate counters and separate collections are used.

7. Migration Process

The migration pipeline is implemented in `migration/migrate.py`. It connects to SQL Server and MongoDB, reads relational rows, transforms them into MongoDB document structures, normalizes values for BSON, and writes them using idempotent upserts keyed by SQL primary keys.

Collection	Migrated Documents
airlines	4
airplanes	5
flights	805
clients	2,005
travel_agencies	3
reservations	12,005
maintenance	303

The migration was rerun with a stable run id to demonstrate idempotency. Because each document is upserted by its SQL ID, repeated runs update existing documents instead of duplicating data.

8. Data Transformations

Collection	Derived/Embedded Field	Description
clients	reservation_count	Number of reservations made by the client.
clients	agency_count	Number of distinct travel agencies used.
clients	total_spent	Sum of ticket prices for the client.
airlines	total_flights	Number of flights operated by the airline.
airlines	total_airplanes	Number of airplanes owned by the airline.
airlines	completed_flights	Number of completed flights.
travel_agencies	total_reservations	Number of reservations created by agency.
travel_agencies	total_ticket_value	Sum of agency ticket prices.
airplanes	embedded airline/model	Airline and model snapshots inside airplane document.

Collection	Derived/Embedded Field	Description
flights	airline_name, flight_duration	Joined airline name and duration in minutes.

9. Error Handling

Error handling is implemented in migration/errors.py, migration/logger.py, and migration/error_demo.py. The required scenarios are malformed source records and database connection failures.

```
Malformed flight source record skipped
Malformed reservation source record skipped
SQL Server connection failure handled
MongoDB connection failure handled
```

10. Validation Results

Validation is implemented in validation/validate.py and validation/checksum.py. It compares record counts, checksums, aggregations, and spot-check queries.

```
PASS count:airlines: SQL=4, MongoDB=4
PASS count:clients: SQL=2005, MongoDB=2005
PASS count:reservations: SQL=12005, MongoDB=12005
PASS checksum:flights:key_fields
Validation summary: 25 passed, 0 failed
```

11. Visualization Layer

The visualization layer is implemented in visualization/charts.py using Plotly. It reads only from MongoDB.

Visualization	Output File	Data Source
Airline performance	visualization/output/airline_performance.html	airlines
Reservation activity	visualization/output/reservation_activity.html	reservations
Travel agency analytics	visualization/output/travel_agency_analytics.html	travel_agencies

12. Docker Support

Docker support is implemented in docker-compose.yml and includes SQL Server 2022 Developer, MongoDB, and Mongo Express.

```
docker compose up -d
docker compose ps
sqlcmd -S localhost -U sa -P "YourStrong!Passw0rd" -C -i sql/schema.sql
python mongodb/setup_collections.py --mongo-uri mongodb://localhost:27017 --database airport_nosql
```

13. Presentation Demo Flow

```
docker compose ps
python migration/migrate.py
python migration/migrate.py --run-id demo-idempotency
python validation/validate.py
python visualization/charts.py
python migration/error_demo.py
```

14. Conclusion

The project successfully implements a complete relational-to-NoSQL migration pipeline. SQL Server is used as the normalized source database with constraints and a large synthetic dataset. MongoDB is used as the denormalized target database with embedded documents and derived fields. The migration is programmatic, idempotent, logged,

and validated. The visualization layer proves that the migrated MongoDB data is usable for analytics.

The final validation passed with 25 checks and 0 failures, confirming that the migration output matches the relational source for counts, checksums, aggregates, and spot checks.